



Pages

In this chapter, you will learn how to declare static pages.

Default pages

In Odoo, websites come with a few default static pages (Home, Contact us, 404, ...). They are built the following way.

```
/website/data/website_data.xml
```

```
<template id="website.homepage" name="Home">
  <t t-call="website.layout"
    pageName.f="homepage">
    <div id="wrap" class="oe_structure oe_empty" />
  </t>
</template>
```

Each default page is a template with its own content saved into a record. This is the reason why, **custom pages are created within a record**.

The `<t-call='website.layout'>` has some variables that can be set:

Define the meta title.

```
<t t-call="website.layout"
  additional_title.translate="My Page Title">
  <div id="wrap" class="oe_structure oe_empty" />
</t>
```

Tip

As the `t-call` accepts parameters, these can be passed as variable, string (`*.f`) or translatable string (`*.translate`).

Define the meta description.

```
<div id="wrap" class="oe_structure oe_empty" />
</t>
```

Add a CSS class to the page.

```
<t t-call="website.layout"
  pageName.f="homepage">
  <div id="wrap" class="oe_structure oe_empty" />
</t>
```

If needed, deactivate default pages.

/website_airproof/data/pages/home.xml

```
<record id="website.homepage" model="ir.ui.view">
  <field name="active" eval="False" />
</record>
```

/website_airproof/data/pages/contactus.xml

```
<record id="website.contactus" model="ir.ui.view">
  <field name="active" eval="False" />
</record>
```

Alternatively, replace the default content of these pages using XPath.

/website_airproof/data/pages/404.xml

```
<template id="404" inherit_id="http_routing.404">
  <!-- Change the meta title parameter through the t-call -->
  <xpath position="attributes" expr="//t[@t-call='web.frontend']">
    <attribute name="additional_title.translate">404 - No page found</attribute>
  </xpath>
  <!-- Replace the default content -->
  <xpath expr="//*[@id='wrap']" position="replace">
    <div id="wrap" class="oe_structure">
      <!-- Content -->
    </div>
  </xpath>
</template>
```

See also

- [Odoo Documentation on SEO](#)

Theme pages

You can add as many pages as you want to your website. Instead of defining a `<template>`, create a page record.

Declaration

/website_airproof/data/pages/about_us.xml

```
<?xml version="1.0" encoding="utf-8"?>
<odoo noupdate="1">
  <record id="page_about_us" model="website.page">
    <field name="name">About us</field>
    <field name="is_published" eval="True" />
    <field name="key">website_airproof.page_about_us</field>
    <field name="url">/about-us</field>
    <field name="website_id" eval="1" />
    <field name="type">qweb</field>
    <field name="arch" type="xml">
      <t t-name="website_airproof.page_about_us">
        <t t-call="website.layout">
          <div id="wrap" class="oe_structure">
            <!-- Content -->
          </div>
        </t>
      </t>
    </field>
  </record>
</odoo>
```

Tip

To set the page as the homepage, set the field below:

Tip

Header and footer are visible by default but can be hidden on specific pages by setting `header_visible` and/or `footer_visible` fields to `false`. This way they are visually hidden but still there in the DOM. Both can be displayed again through the Website Builder like any hidden elements hidden under a **Visibility Condition**.

```
<field name="header_visible" eval="False" />
<field name="footer_visible" eval="False" />
```

The Header can be hidden on all pages by enabling this view (See also: **Presets**):

/website_airproof/data/presets.xml

```
<record id="website.option_layout_hide_header" model="ir.ui"
  <field name="active" eval="True" />
</record>
```

Multiwebsite and website_id

In a module context, the record created above is available, by default, on every website available on the database. It's preferable to specify the `website_id` of the website where the page will be findable.

Field	Description
name	Page name (human-readable).
is_published	Define if the page is published (visible to visitors).
key	View key (must be unique)
url	Relative path where the page is reachable.
type	View type

With `<t t-call="website.layout">` you use the Odoo default page layout with your code.

noupdate attribute

This attribute prevents data overwriting. It can be added either on a `data` tag wrapping some records to protect or on the `odoo` tag in order to protect all records declared into the file.

Protect all records of the file:

```
<?xml version="1.0" encoding="utf-8"?>
<odoo noupdate="1">
  <record id="menu_company" model="website.menu">
    <!-- Fields -->
  </record>
  <record id="menu_faq" model="website.menu">
    <!-- Fields -->
  </record>
</odoo>
```

Protect specific records in the file:

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <record id="menu_company" model="website.menu">
    <!-- Fields -->
  </record>

  <data noupdate="1">
    <record id="menu_faq" model="website.menu">
      <!-- Fields -->
    </record>
    <record id="menu_legal" model="website.menu">
      <!-- Fields -->
    </record>
  </data>
</odoo>
```

Use case

Problem

In case of a module update on the database, every record declared into the module will overwrite those existing in the database even if the end-user has changed some of these records.

Solution

By wrapping the record (or all records declared in the file) into a `<data noupdate="1"></data>` tag, the record declared is created at the first module installation but not updated after a module update.

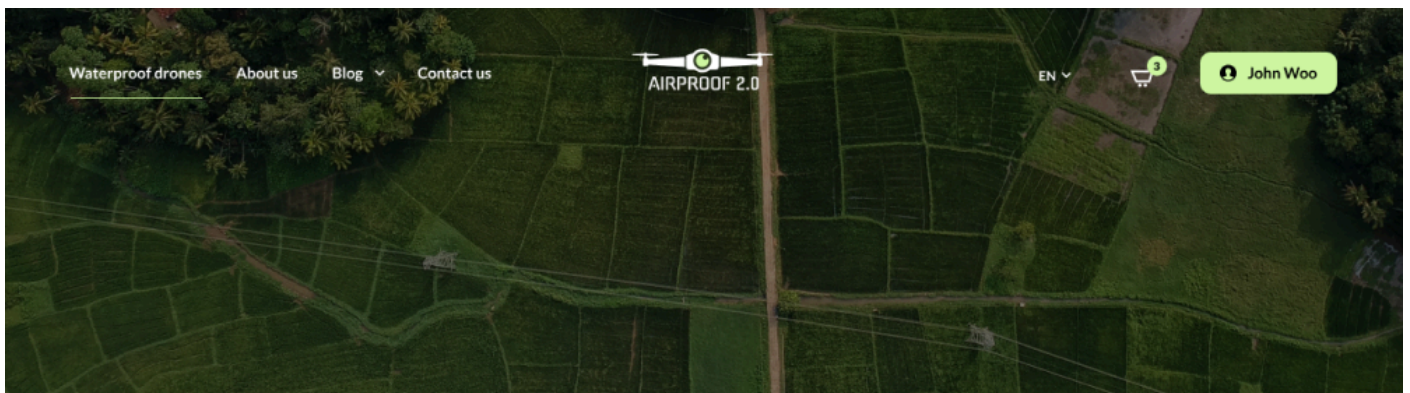
What happens if the record has been manually deleted (e.g.: a menu item) ?

Is this method only valid on static page records?

Header overlay

Make the header background transparent and stand on top of the page content.

```
<field name="header_overlay" eval="True"/>
```



Note

To create the content of a static page, use the Odoo way of doing things in order to remain editable by the Website Builder. Please note that Odoo takes advantage of

- [Bootstrap documentation](#)

Page templates

Create preset static page templates available from the New Page dialog window.

Declaration

The page templates has to be defined into the `__manifest__.py` of the module through `new_page_templates` and `new_page_template_templates.xml` :

`/website_airproof/__manifest__.py`

```
{
    'name': 'Airproof Theme',
    'description': '...',
    'category': 'Website/Theme',
    'version': '19.0.0.0',
    'author': '...',
    'license': '...',
    'depends': ['website'],
    'data': [
        # ...
        'views/new_page_template_templates.xml'
    ],
    'assets': {
        # ...
    },
    'new_page_templates': {
        'airproof': {
            'faq': ['s_airproof_text_block_h1', 's_title', 's_
        }
    }
}
```

Templates

Then you have to create the template using a specific naming convention based on the hierarchy into the `__manifest__.py` . In this case, the name is

apply some adaptations:

/website_airproof/views/new_page_template_templates.xml

```
<template id="s_airproof_text_block_h1" inherit_id="website
  <xpath expr="//div[class('container')]|//div[class
    <div class="container s_allow_columns">
      <h1 class="display-1">FAQ - Help</h1>
    </div>
  </xpath>
</template>
```

Instantiate each building block (modified or not) for the page template:

/website_airproof/views/new_page_template_templates.xml

```
<template id="new_page_template_airproof_faq_s_text_block_h
<template id="new_page_template_airproof_faq_s_title" inher
<template id="new_page_template_airproof_faq_s_faq_collapse
<template id="new_page_template_airproof_faq_s_call_to_acti
```

Then, create your page template with some `t-snippet-call` within an `#wrap` as explained above:

/website_airproof/views/new_page_template_templates.xml

```
<template id="new_page_template_sections_airproof_faq" name:
  <div id="wrap">
    <t t-snippet-call="website_airproof.new_page_template
    <t t-snippet-call="website_airproof.new_page_template
    <t t-snippet-call="website_airproof.new_page_template
    <t t-snippet-call="website_airproof.new_page_template
  </div>
</template>
```

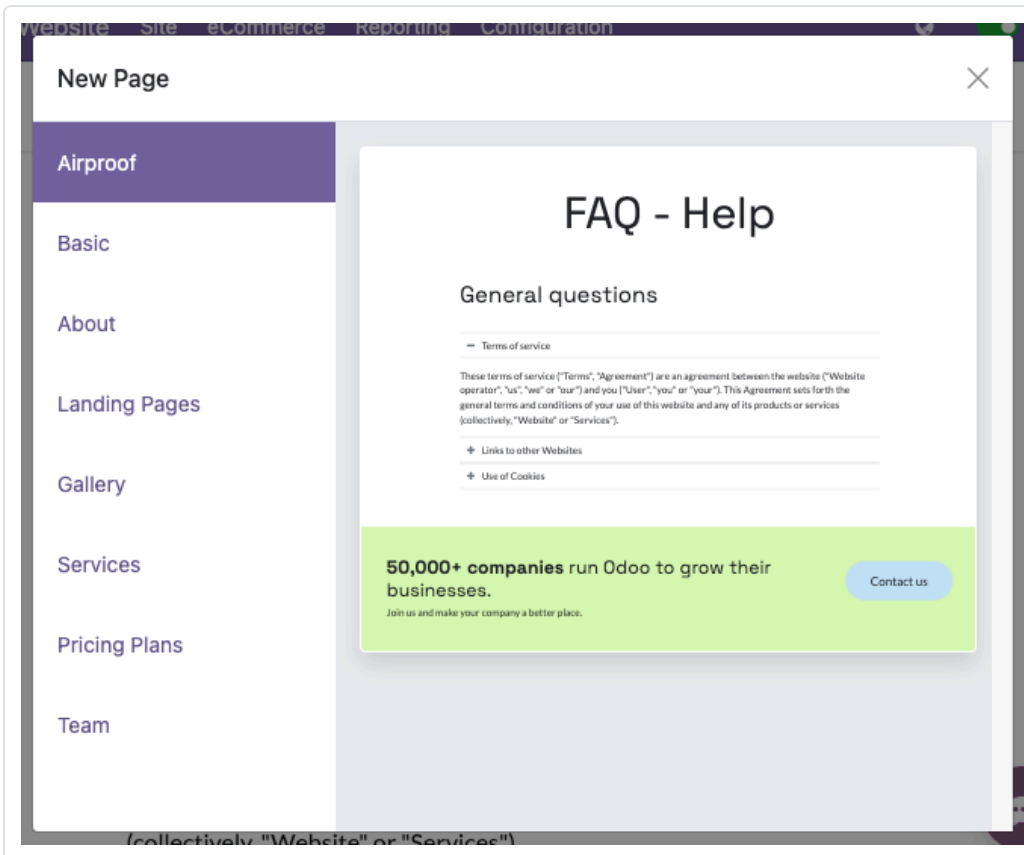
Once the page template is created, create a custom group and add it to the existing ones. Find below a list of the existing groups:


```
<div id="about">About</div>
<div id="landing">Landing Pages</div>
<div id="gallery">Gallery</div>
<div id="services">Services</div>
<div id="pricing">Pricing Plans</div>
<div id="team">Team</div>
</template>
```

Feel free to add custom groups to the list:

/website_airproof/views/new_page_template_templates.xml

```
<template id="new_page_template_groups" inherit_id="website
  <xpath expr="//div[@id='custom']" position="after">
    <div id="airproof">Airproof</div>
  </xpath>
</template>
```





Get Help

Contact Support

Ask the Odoo Community

